

AASD 4010

Deep Learning - I

Applied AI Solutions Developer Program



BERTology



Agenda



BERT
BERT Input & Output
Applications of BERT
BERT Architecture
Pre-Training Tasks



BERT Vs RNN

Prior to BERT and Transformer model, LSTMs have **Recurrence**

A RNN looks at the words in a sentence one at a time, and gradually builds up the understanding of the sentence

Input: Current word + hidden state from processing of previous word

Recursiveness complicates the model (backpropagation through layers and time steps). Complex

No Recurrence in BERT



Pre-Training

Pre-Training of BERT involves

- Masked Language Model (**MLM**)
- Next Sentence Prediction (**NSP**)

MLM --> Predict the masked word

NSP --> Is Sentence B follow Sentence A logically?

Fake Tasks – BERT is trained to perform these tasks to force it to develop an amazing understanding of the language. Once BERT is trained, discard the tiny piece of the model and replace it with something useful

MLM & NSP **do not** need labeled data



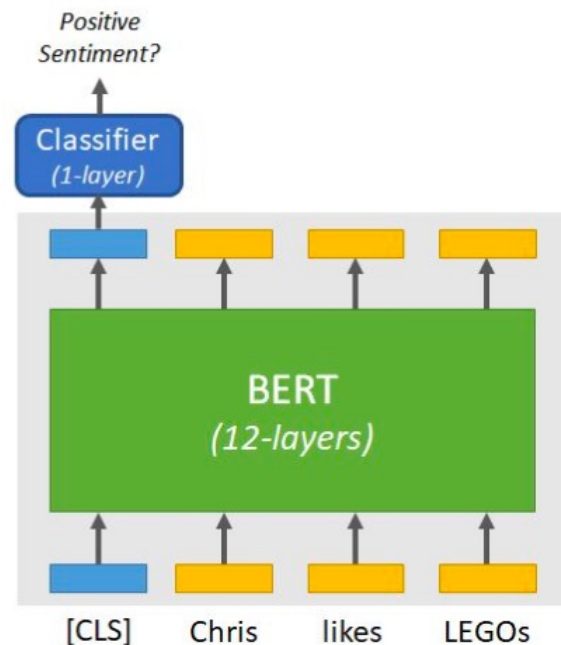
Fine-Tuning

BERT – 12 layer NN

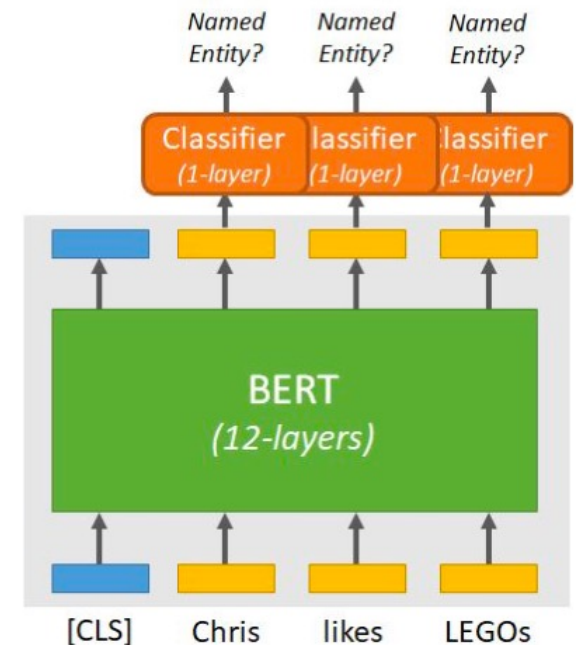
MLM & NSP adds a small, single-layer classifier to get outputs

Magic – You can replace this final layer with your own task-specific model leveraging all BERT's knowledge

Sentiment Analysis



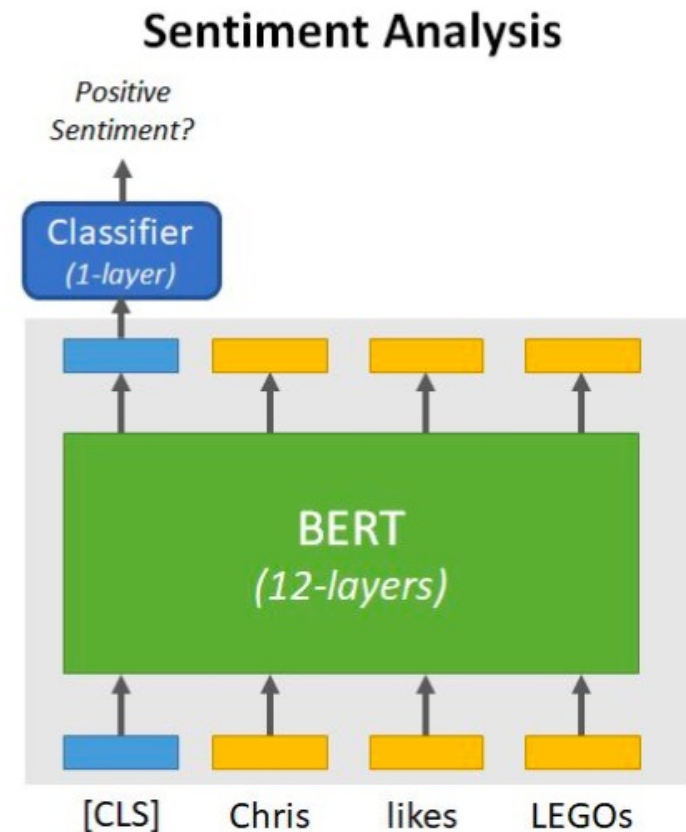
Named Entity Recognition



Sentiment Analysis

Text Classification Task – Sentiment Analysis

1. Take a pretrained BERT
2. Use only the CLS token embedding output
3. Add an untrained layer of neurons on the end
4. *Train the new, combined model on the custom dataset and task – **Fine tuning***

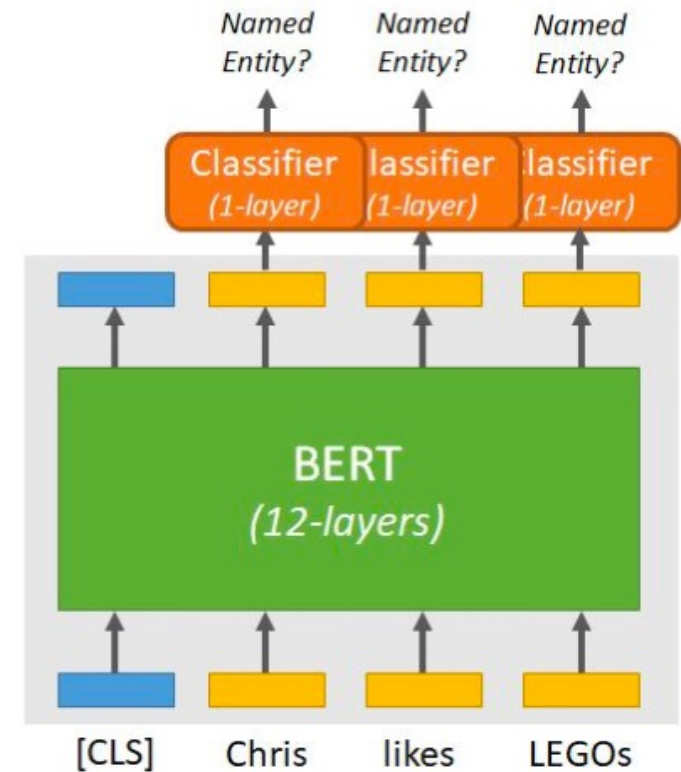


NER

Token Classification Task – NER

1. Take a pretrained BERT
2. Use all output but CLS output
3. Add an untrained layer of neurons on the end for every input word
4. *Train the new, combined model on the custom dataset and task – **Fine tuning***

Named Entity Recognition



Transfer Learning

Transfer Learning = Pre-Training + Fine-Tuning

Advantages

- Quick Dev time
- Less data
- High Accuracy



BERT Input & Output

BERT Tokenizer

Pros

Takes any arbitrary text and breaks it into "BERT" like tokens
Handles even Out-of-Vocab words

Cons

Need to **ONLY** use BERT embeddings, **NO** custom embeddings



Out-of-Vocab words

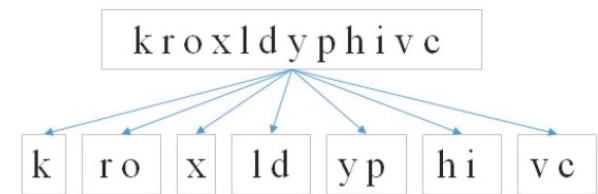
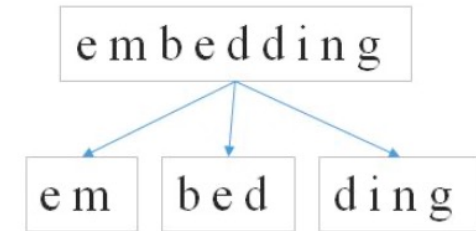
BERT Tokenizer uses "WordPiece" model

If a word in tokenization process is not available in the vocabulary, it breaks down into subwords or word pieces

30,000 tokens

60-80 % total words

Rest is subwords or word pieces



BERT Character set - 997

[illegible]

Smaller Vocabulary

Rarer words will have very less training data

By word pieces, you can re-use word pieces of the mostly occurring "flabbergasted" to encode less occurring "flabbergast" and "flabbergasting"

Flabbergast = flabber + gast

Flabbergasting = flabber + gast + ing

Also, word associations and relationships are maintained

Form	Count
flabbergast	10
flabbergasts	0
flabbergasted	218
flabbergasting	12



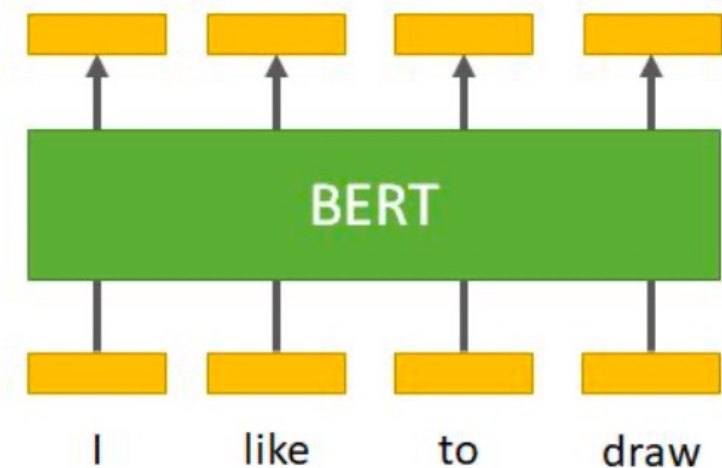
Tokenizer

When BERT is applied to a piece of text

- Tokenizer will split text into tokens
- Looks up embeddings for tokens
- Outputs the embeddings (768 dimension)

Outputs:
"Enhanced"
Embeddings

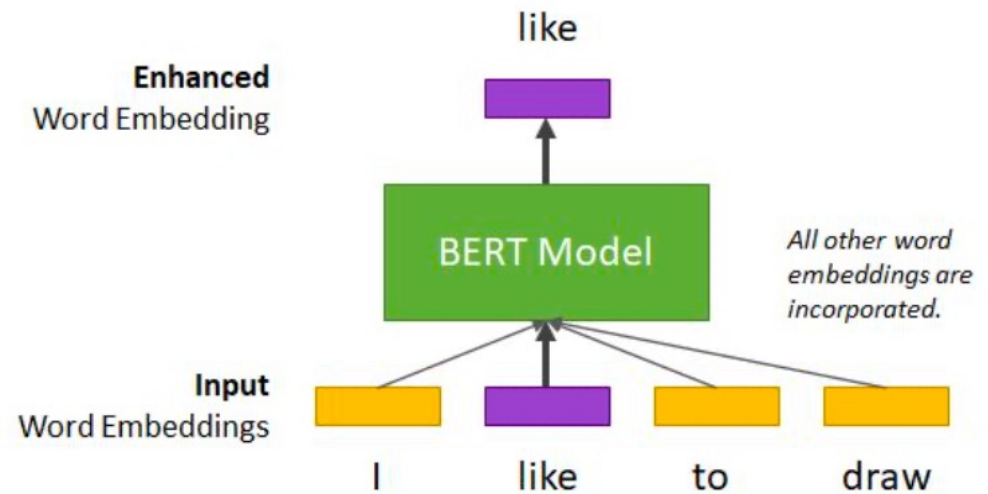
Inputs:
Word
Embeddings



Enhanced Embedding

When Embedding "**like**",

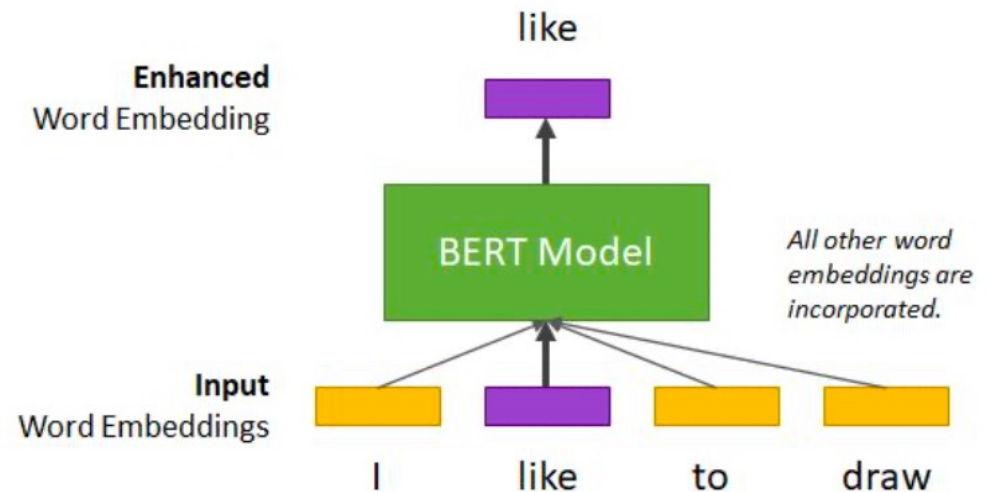
- Like embedding is fed
- Also, context (other words' embeddings are also fed)
- *Supplemental info*



Weighted Averaging for Variable Length

To counter **variable lengths**,

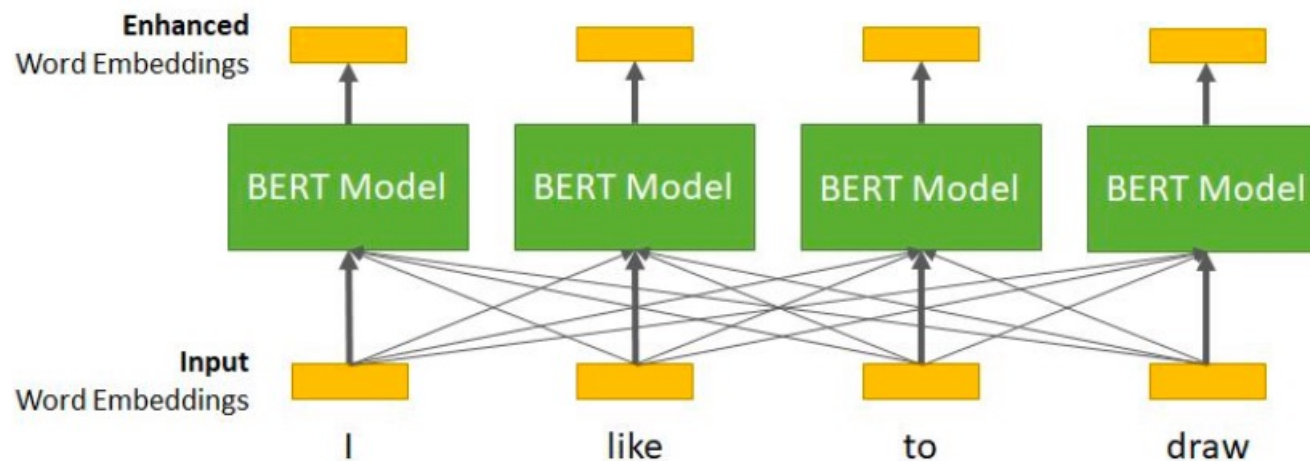
- Weighted average of the input vectors
- Used in "**Self-Attention**" layer



Parallelization

BERT processes each word **independently**,

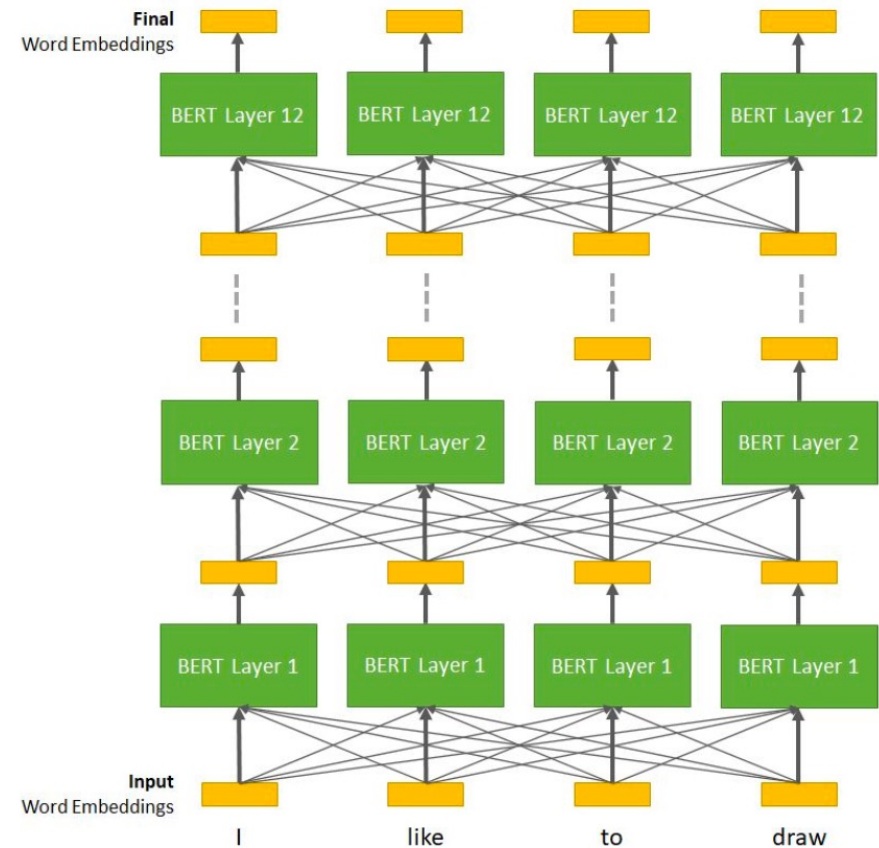
- For four words to process, four copies of BERT model is used



12 Layers

BERT processes each word independently through these **12 layers**,

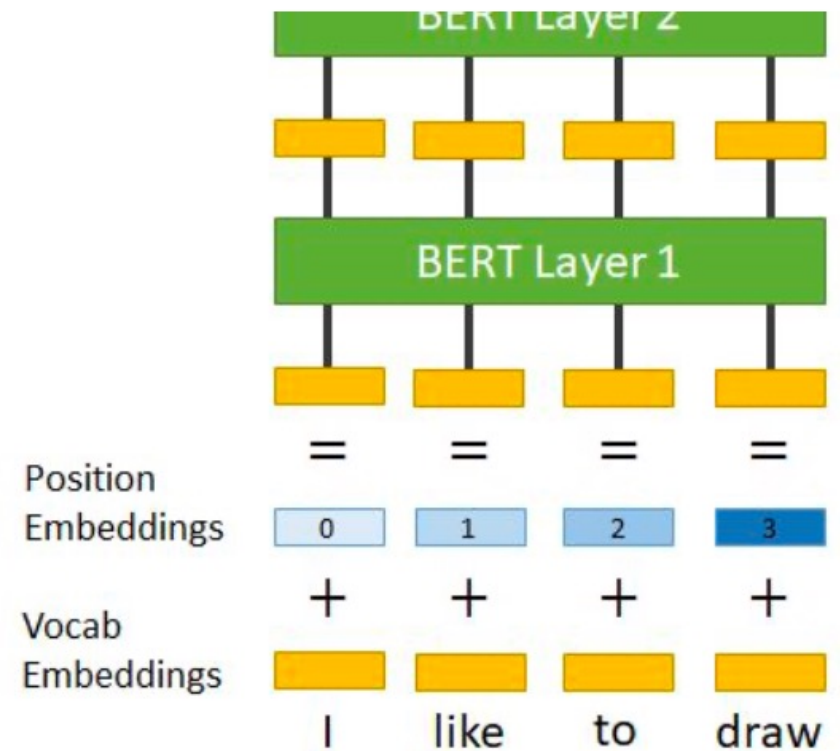
- Every layer **enhances** the embedding of the word with its context
- Every other upper layer does the same till the final layer
- All this process is happening **parallel at one layer** (at once all words are seen and embeddings are enhanced and passed to the above layer)



Positional Encoding (PE)

As BERT can process words in any order in parallel,

- Embedding of token is added with **PE vector**
- Input vector to BERT layer is now token embedding added with Positional embedding (**position of that word in the sentence**)
- Max = **512 tokens**



BERT Applications

Applications

- Token Classification
 - NER
- Text Classification
 - Sentiment Analysis
- Text Pair Classification
 - Question Answering

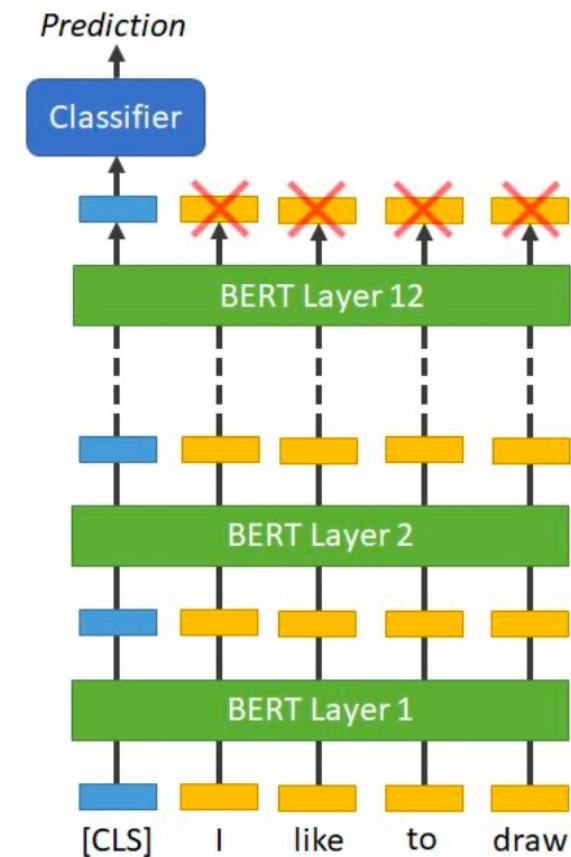


Text Classification

When BERT is fed an input text,

- First token is a **[CLS]** token
- This token is propagated to other layers until the last layer
- Add a single-layer linear classifier only taking this [CLS] BERT embedding as input and providing a classification output
- All other token embeddings are **omitted**

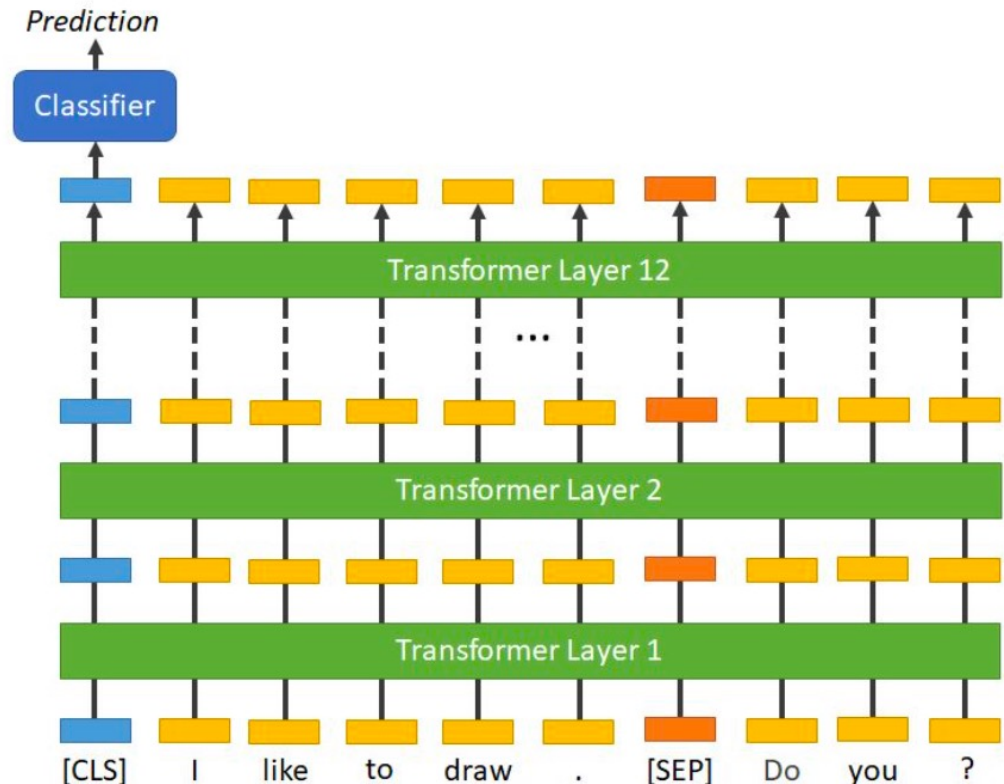
Note: [CLS] token would've been enhanced by the other words (context) in all 12 layers



Text Pair Classification

When BERT is fed an input text,

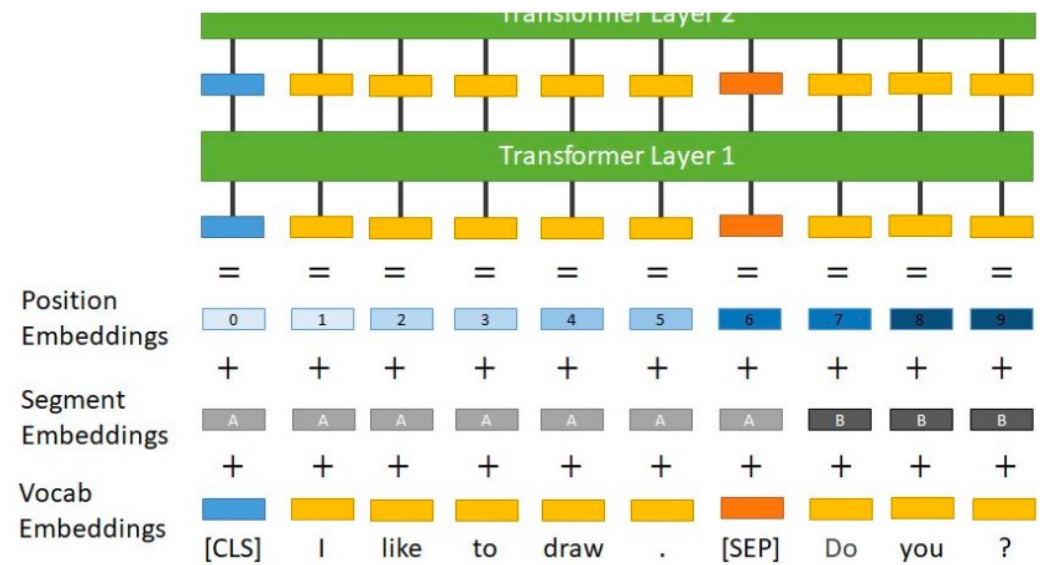
- First token is a **[CLS]** token
- **[SEP]** token is sent as sentence separator
- Add a single-layer linear classifier only taking this [CLS] BERT embedding as input and providing a classification output
- All other token embeddings are **omitted**



Text Pair Classification

When BERT is fed an input text,

- First token is a **[CLS]** token
- **[SEP]** token is sent as sentence separator
- **Segment embeddings** are added for every sentence
- **Positional Embeddings** are added for every word
- Add a single-layer linear classifier only taking this [CLS] BERT embedding as input and providing a classification output
- All other token embeddings are **omitted**



BERT is not good

Applications

- Text Generation
 - Text Translation
 - Speech Recognition
 - Auto-Completion
-
- Transformer included "component" decoder for generating text, but this had to be removed in BERT
 - Transformer had encoder + decoder to translate text
 - BERT = Transformer – Decoder



BERT vs OpenAI-GPT

OpenAI GPT used Language Modeling for its pre-training process

- Predict the next word is the task
- Feed one word at a time, generate next word
- Will see only words to the left

BERT pre-training tasks of MLM and NSP

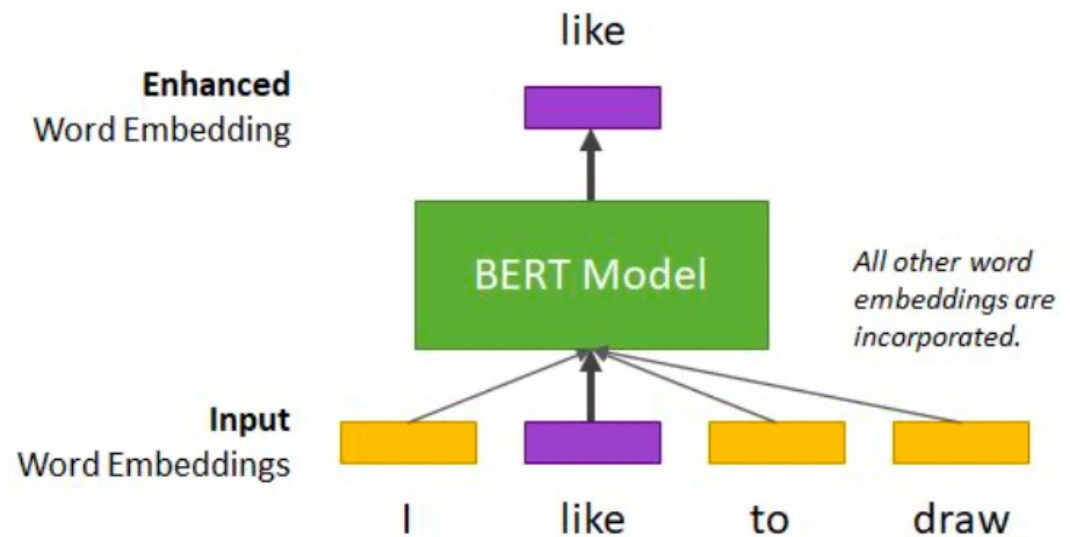
- Give BERT the entire input sequence at once
- BERT will factor in all of the words at once to get to its predictions
- Will have all the words to be used for predicting (bidirectional)



Self-Attention

How the BERT model **enhances** the embedding?

Key
Query
Value



Self-Attention

What does the word "**he**" refer to?

Bert

Ernie

The hat



Bert notices the hat, but chooses not to engage Ernie since he knows it'll lead to some ridiculous stuff.



Weighted Average in Self-Attention

When producing "enhanced embedding" of word "he",

- Weighted-average of the embeddings of other context words are taken into consideration
- How much **attention** to give to that context word

Bert	notices	the	hat,	but	chooses	not	to	engage	Ernie
since	he	knows	it'll	lead	to	some	ridiculous	stuff.	



Weighted Average in Self-Attention

Multiply each embedding value by its weight

Sum it up for all positions of the input

Divide the sum by sum of the weights

$$\bar{x} = \frac{\sum_{i=1}^n w_i x_i}{\sum_{i=1}^n w_i}$$

Bert	notices	the	hat,	but	chooses	not	to	engage	Ernie
since	he	knows	it'll	lead	to	some	ridiculous	stuff.	



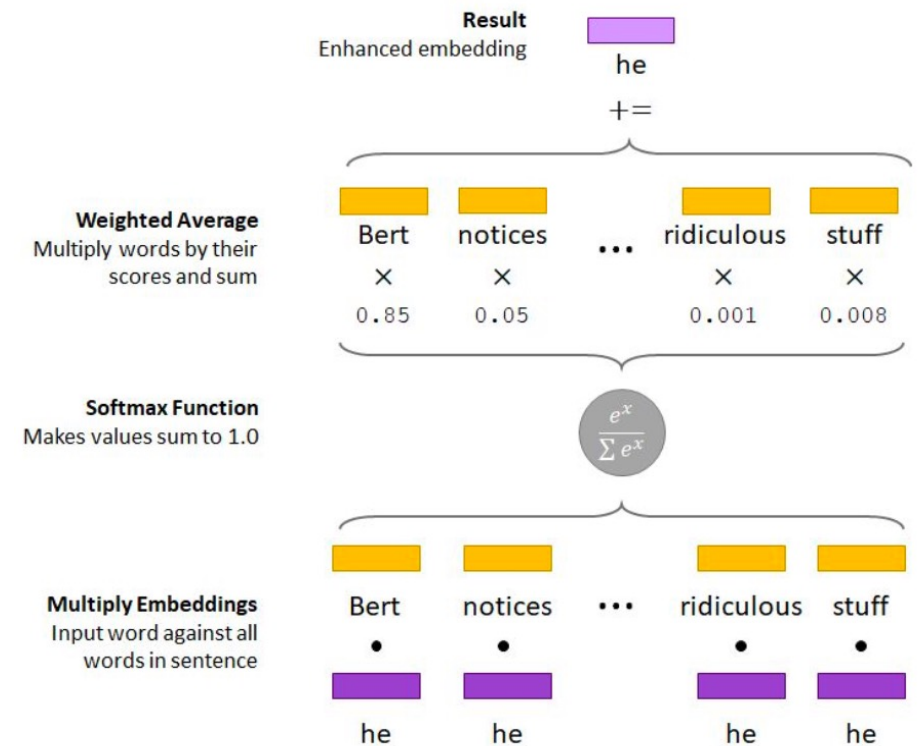
Bert notices the hat, but chooses not to engage Ernie since he knows it'll lead to some ridiculous stuff.



Calculation of Attention Weights

To compute Attention weights,

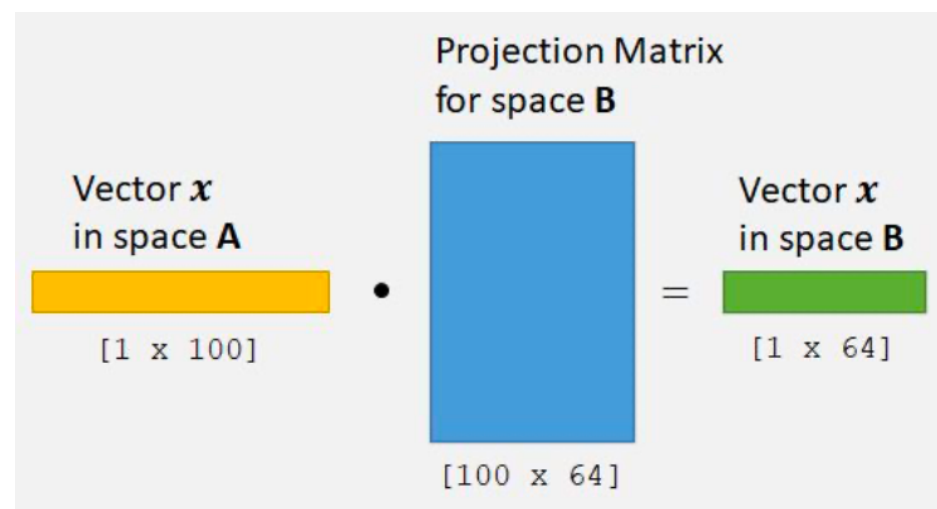
- **Dot product** of "he" embedding and "Bert" embedding to give it to "Bert" embedding
- Do this for all context words and feed them through **softmax function** and get the weights



Projection of embeddings

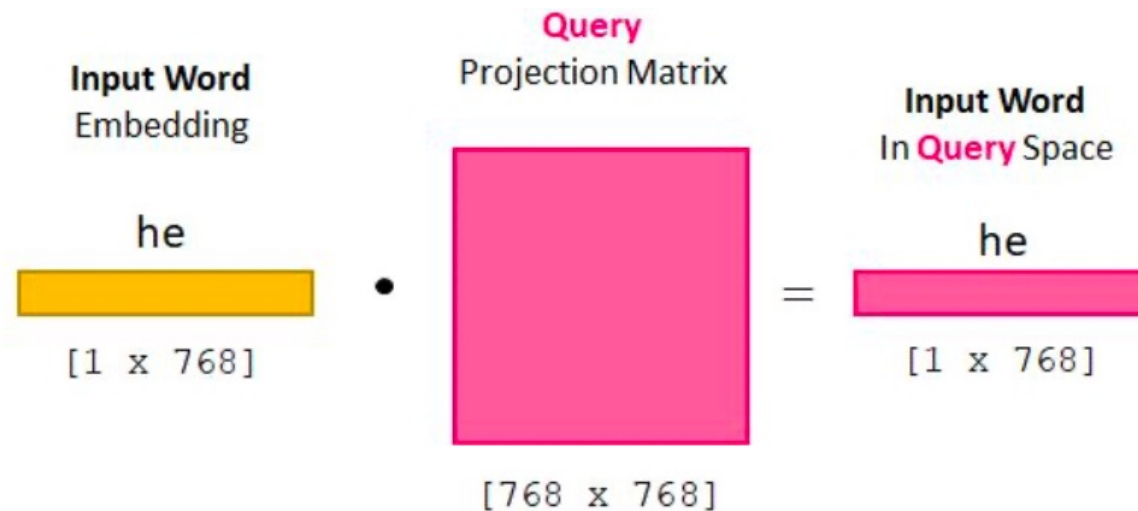
Projection in ML

- **Vector** of an object
- Multiply against a **projection matrix**
- To get a vector in **different** vector space
- In BERT, we are going to project each word embedding onto **3 different vector spaces**
 - Query
 - Key
 - Value



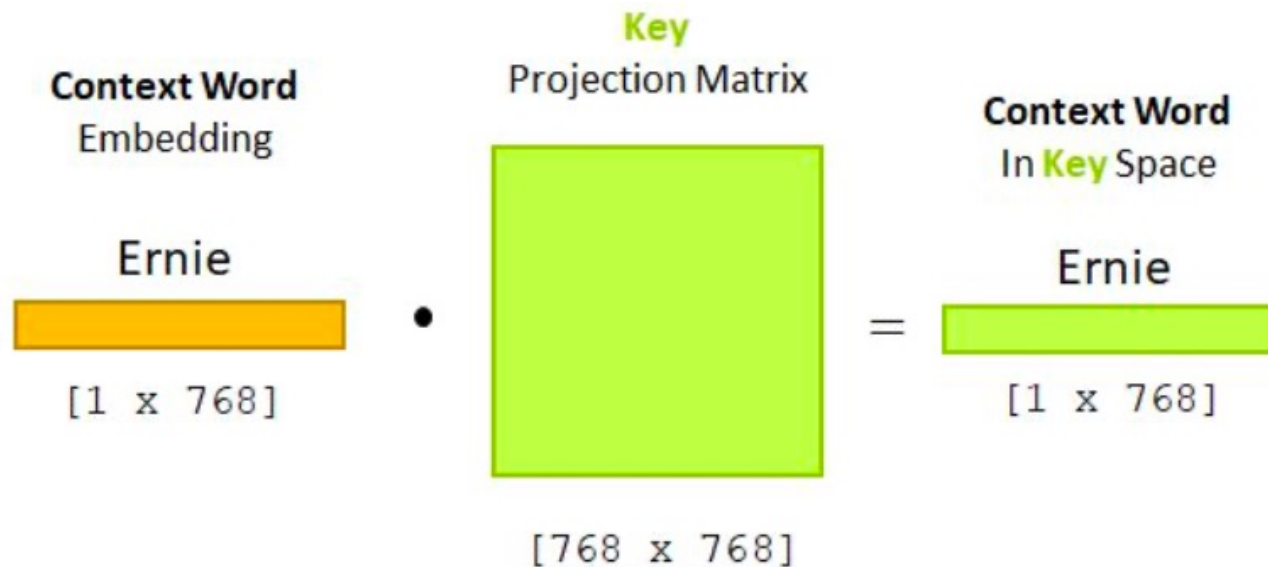
Query Space

Project **input word** "he" into **Query space**



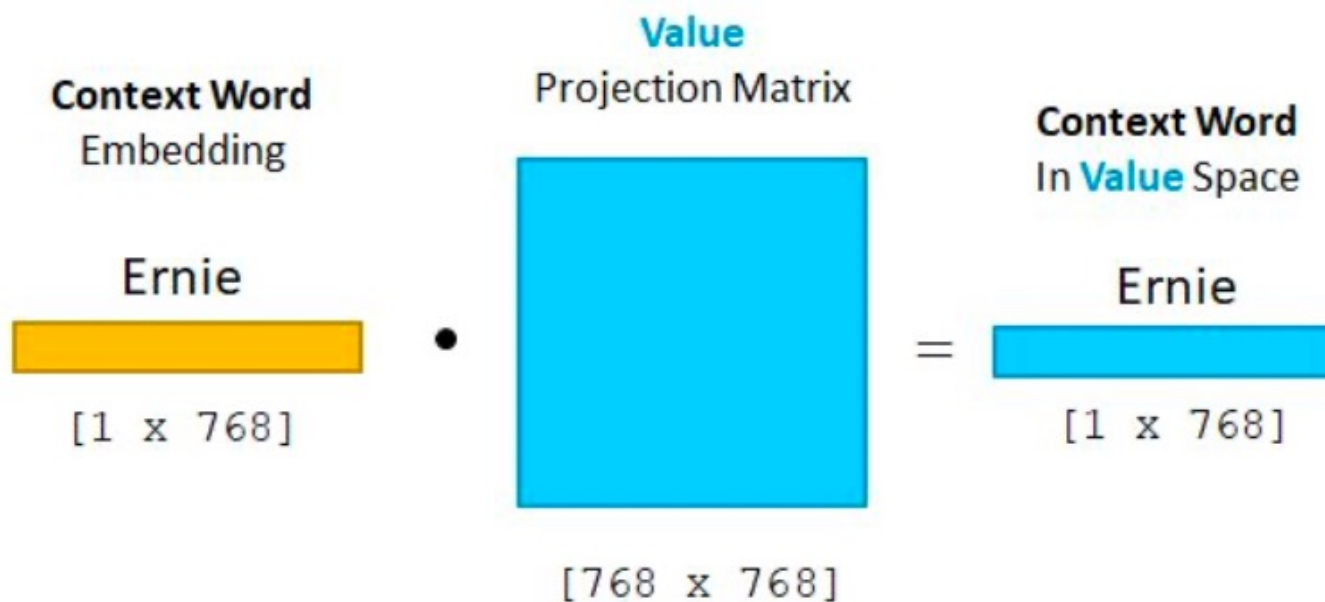
Key Space

For the dot-product step, we will project every **context word** onto **Key space**



Value Space

For weighted-average step, project every **context word** onto **Value space**

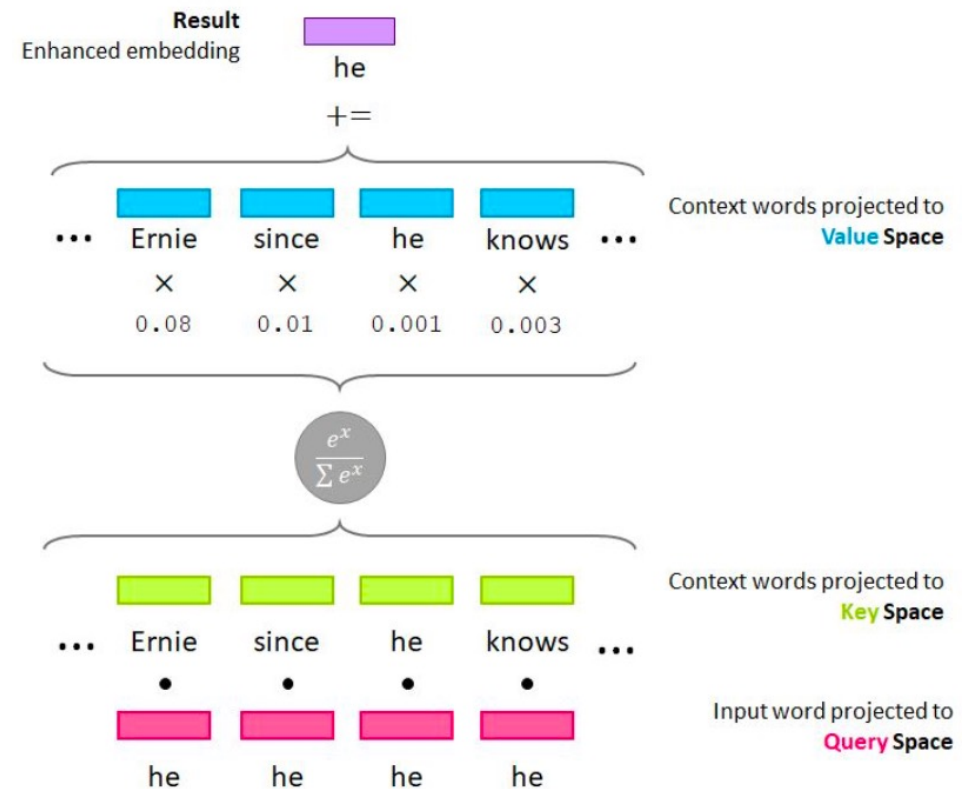


Query, Key, Value Vector Spaces

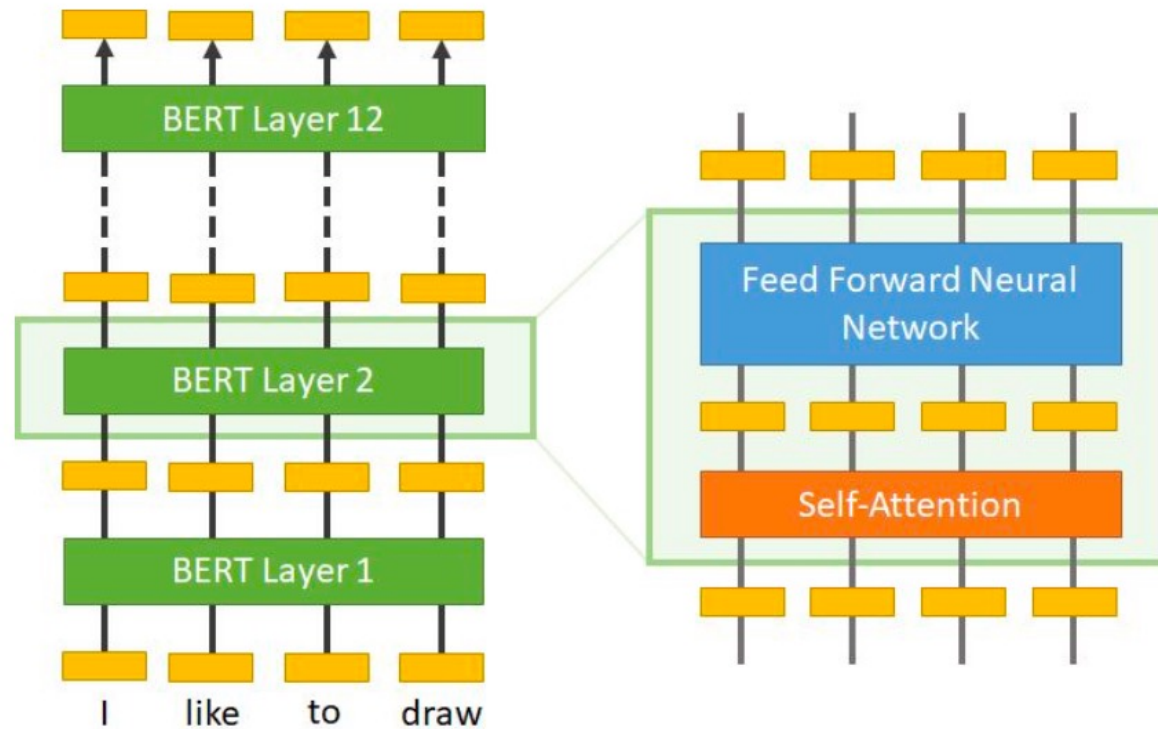
Project **input word** "he" into **Query space**

For the dot-product step, we will project every **context word** onto **Key space**

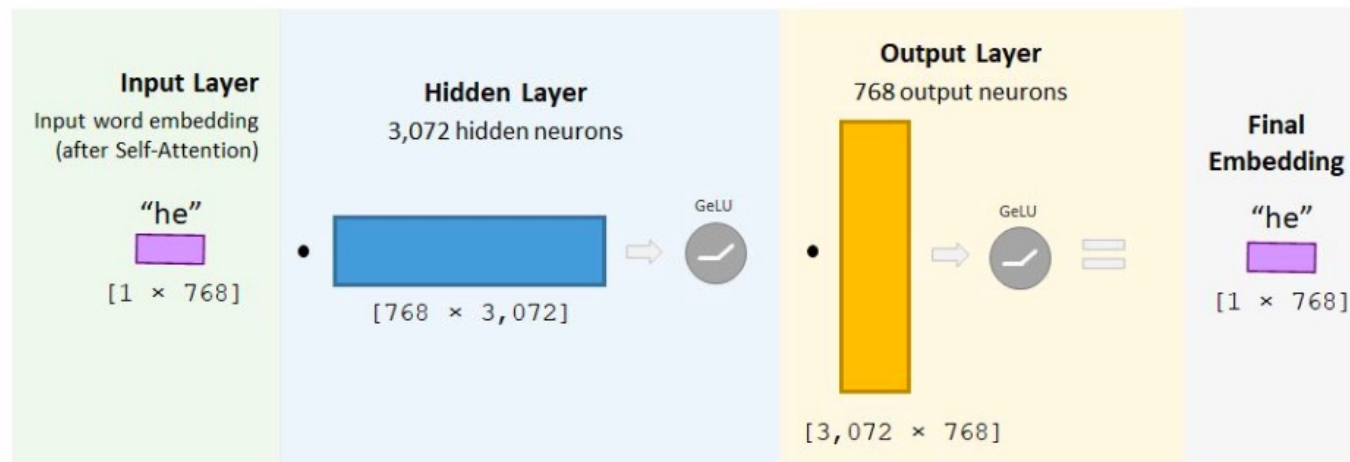
For weighted-average step, we will project every **context word** onto **Value space**



Feed-Forward Neural Network



Feed-Forward Neural Network



Input: 768 dimension input vector (word embedding after Self-attention)

Output: 768 dimension output vector

Hidden layer: 3072 neurons (4x embedding size)

Output layer: 768 neurons

Activation function: GeLU (Gaussian Error Linear Unit)

<https://medium.com/@shoray.goel/gelu-gaussian-error-linear-unit-4ec59fb2e47c>



Weights in BERT

Self-Attention

3 Projection matrices of 768 x 768
 $3 \times 768 \times 768 = 1,769,472$ weights ~ **1.8 M**

FFNN

Hidden layer and output layer 768 x 3078
 $2 \times 768 \times 3078 = 4,718,592$ weights ~ **4.7 M**

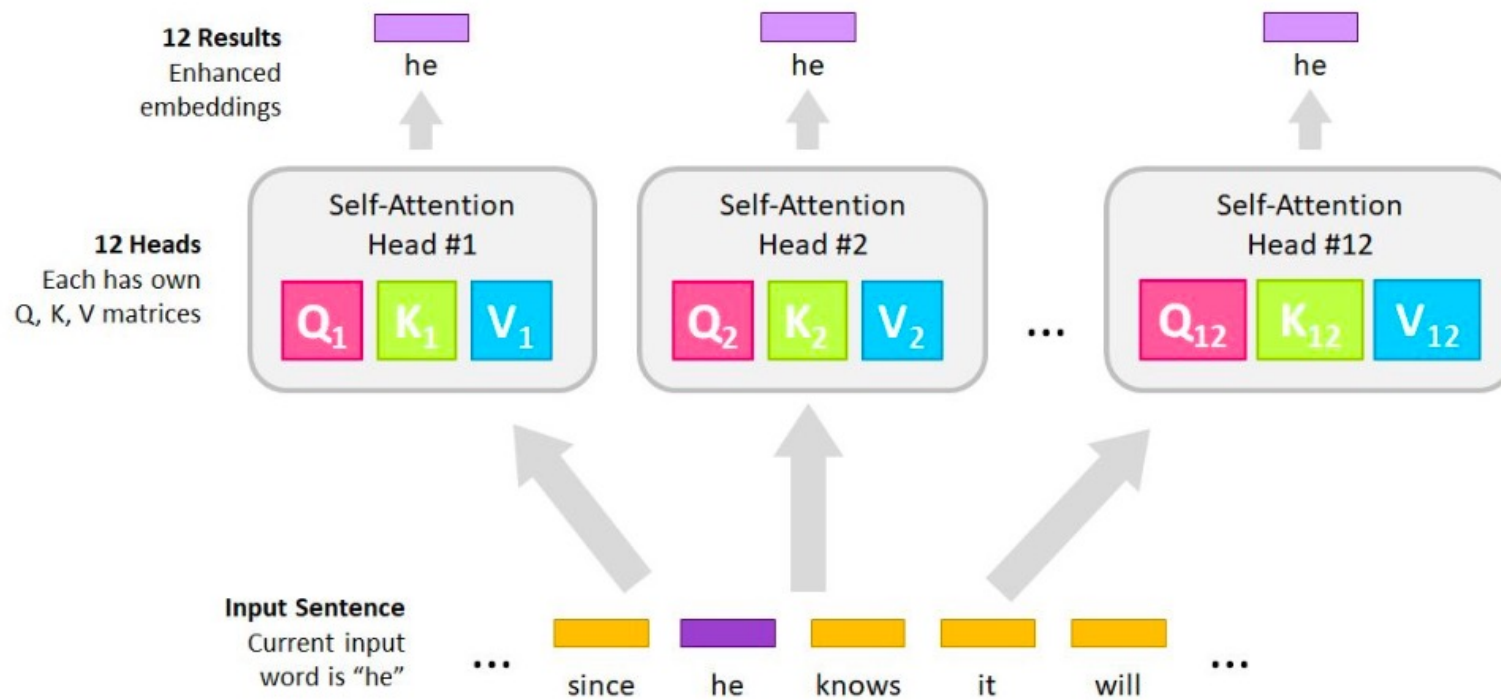
1 BERT layer weights ~ **6.5 M**

12 layers = 6.5×12 ~ **78 M**

Component	# of Weights	% of Total
Self-Attention	~1.8M	27%
Feed Forward Neural Network	~4.7M	73%
TOTAL	~6.5M	100%



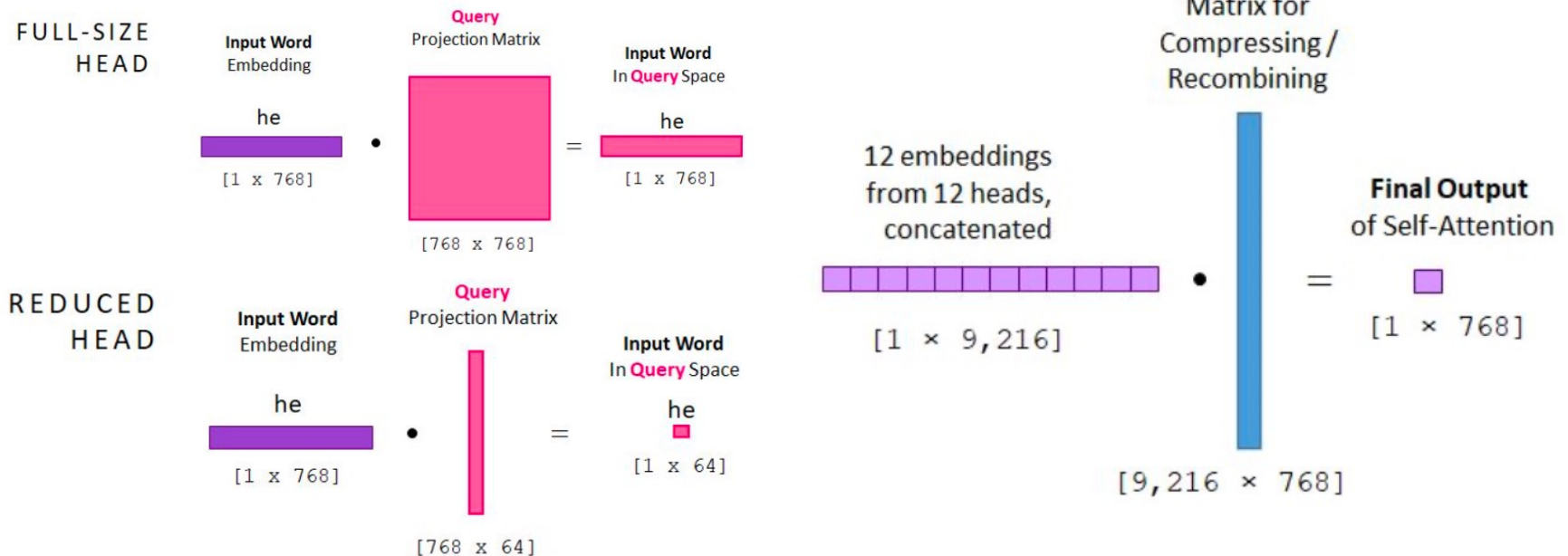
Multi-headed Attention



Multi-headed Attention

Too much of weights in 12 heads

Reduce the size of Key, Query, Value spaces



Pre-Training in BERT

Training Set

- BookCorpus
 - 11,038 books from the web
- Wikipedia

Property	Value
# of books	11,038
# of sentences	74,004,228
# of words	984,846,357
# of unique words mean	1,316,420
# of words per sentence median	13
# of words per sentence	11

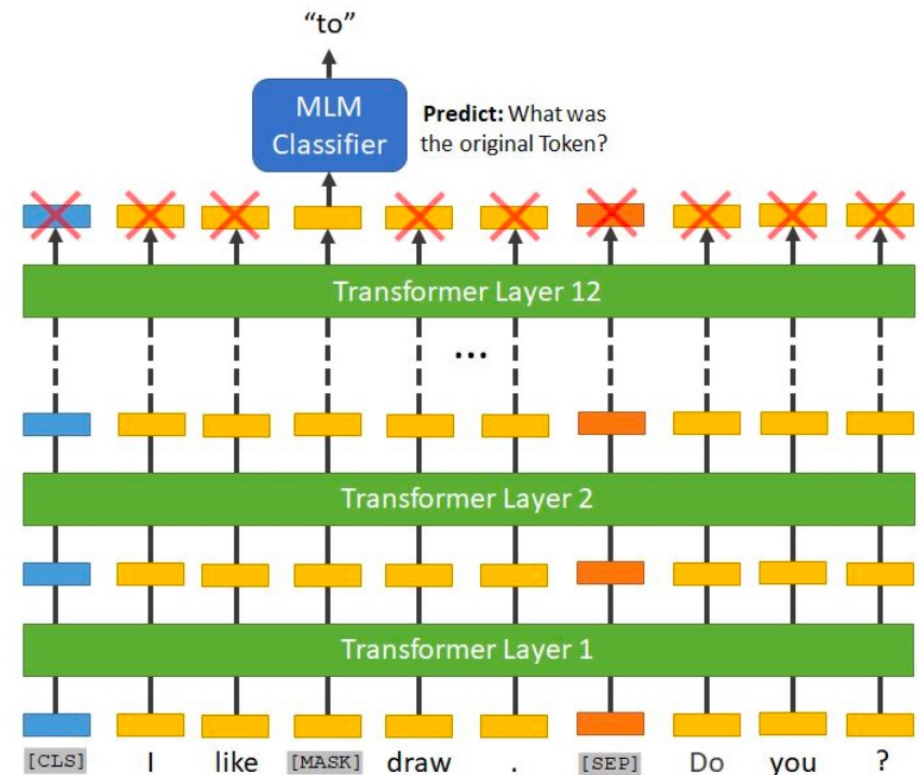
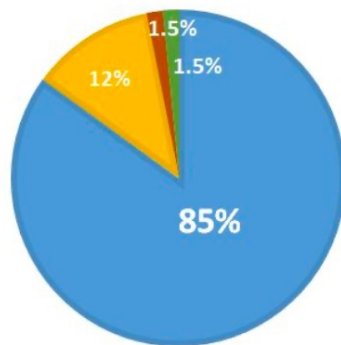


Masked Language Model (MLM)

15 % masked words

- 12% are masked
- 1.5 % are replaced
- 1.5 % are unchanged

■ Untouched ■ Masked ■ Swapped ■ Swapped-Same



Masked Language Model (MLM)

this is not creative . those are the dictionary definitions of the
 this is not creative . those are [MASK] dictionary definitions of the

terms insurance and en ##sur ##ance as properly applied to destruction
 terms [MASK] and en ##sur ##ance as [MASK] applied to destruction

. if you don ' t understand that , fine , legitimate criticism
 . if you don [MASK] [MASK] understand [MASK] , fine want [MASK] criticism

, i ' ll write up three man cell and bounty hunter and then
 , [MASK] ' ll write up three man [MASK] and [MASK] hunter and then

it will be easy to understand why ensured and ins ##ured are different
 it will be easy , understand why ensured and ins ##ured are different

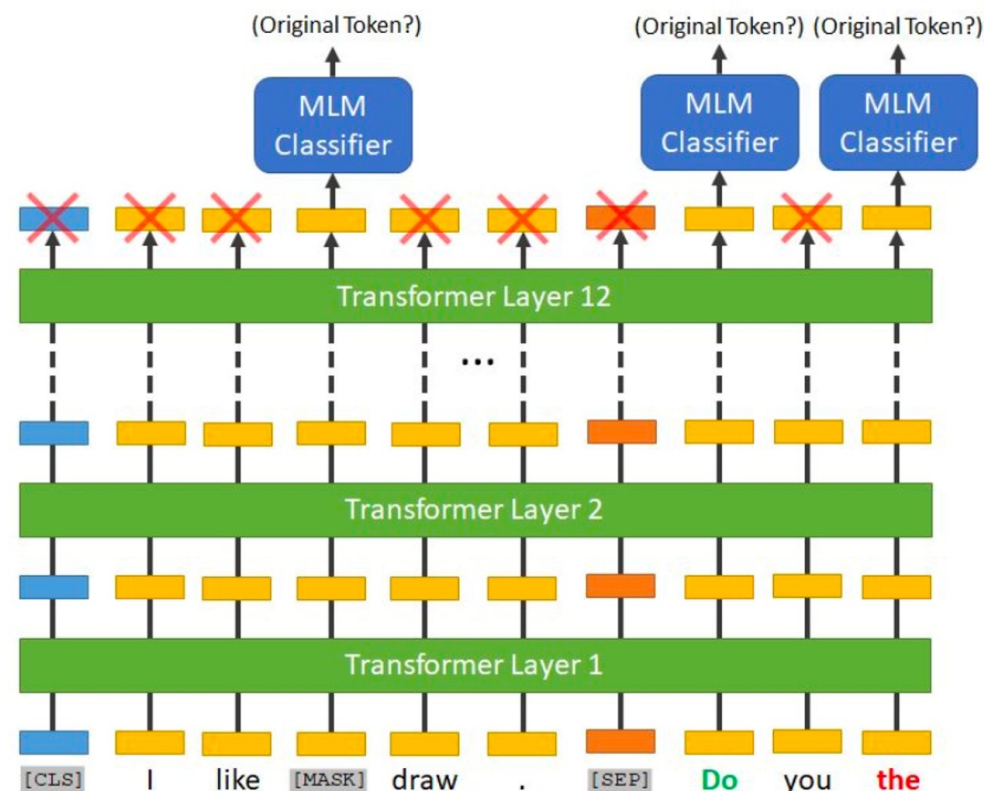
- and why both differ from assured . the sentence you quote is absolutely
 - and why [MASK] differ from assured . the [MASK] you quote is absolutely



Masked Language Model (MLM)

15 % masked words

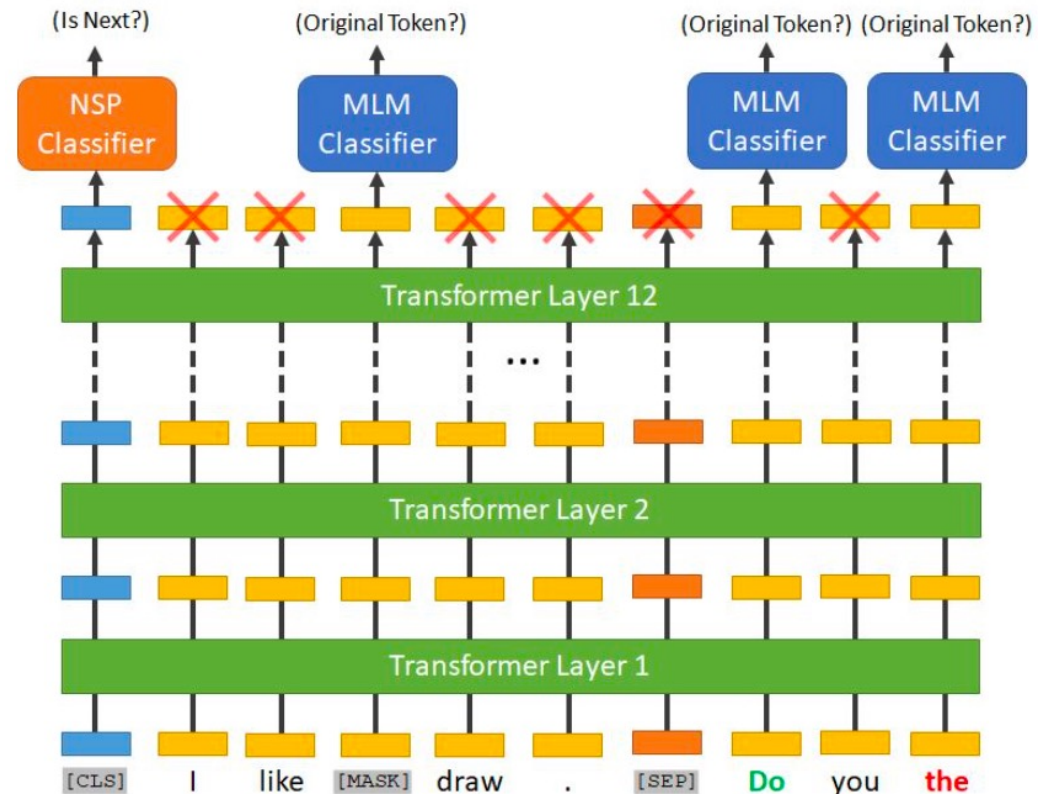
- 12% are masked
- 1.5 % are replaced
- 1.5 % are unchanged
- **[MASK]** words needs to be predicted by BERT only by looking at their **context words**



Next Sentence Prediction (NSP)

Text pairs

- 50% second sentence logically follows the first sentence
- **[SEP]** separates first sentence from second sentence
- **[CLS]** is used to make prediction



Improvements

- **Whole word masking**
 - ALBERT masks 1-3 whole words to make it too difficult for the model to pre-train
- **NSP replaced**
 - RoBERTa , XLNet
- **Sentence Order Prediction (SOP)**
 - **ALBERT** reverses the sentence pairs' order and asked the model to predict if the two pairs are in correct order
- **Plausible Substitutions**
 - **ELECTRA** used "Generator Network" to replace words with a "plausible" alternative and asked the model to predict if it is swapped or not



Further Reading

BERT

<https://arxiv.org/pdf/1810.04805.pdf>

<https://github.com/google-research/bert>

ALBERT

<https://arxiv.org/pdf/1909.11942.pdf>

XLNET

<https://arxiv.org/pdf/1906.08237.pdf>

RoBERTa

<https://arxiv.org/pdf/1907.11692.pdf>

Jay Alammara Illustrations

Chris McCormick Youtube Channel

